

# Data in the Cloud

# Outline

- Data Sources
- Data Analysis
- Update Intensive Workloads
- NoSQL
- Key Value Stores
- Data Consistency
- Benchmarking Serving Systems

# Data Sources

- Science
  - Data bases from astronomy, genomics, environmental data, transportation data, ...
- Humanities and Social Sciences
  - Scanned books, historical documents, social interactions data, ...
- Business & Commerce
  - Corporate sales, stock market transactions, census, airline traffic, ...
- Entertainment
  - Internet images, Hollywood movies, MP3 files, ...
- Medicine
  - MRI & CT scans, patient records, ...

# Data Sources (cont.)

- Data capture and collection:
  - Highly instrumented environment
  - Sensors and Smart Devices
  - Network
- Data storage:
  - High capacity storage media such as disks.



# What can we do with this data wealth?

- What can we do?

- Scientific breakthroughs
- Business process efficiencies
- Realistic special effects
- Improve quality-of-life: healthcare, transportation, environmental disasters, daily life, ...

- Could We Do More?

- YES: but need major advances in our capability to analyze this data

# Cloud Computing approach

**“We have terabytes of click-stream data – what can we do with it?”**

- Very large data repositories
- Complex analysis
- Distributed and parallel data processing
- Complex data processing – Graphs and beyond
- Multidimensional Data Analytics: Location-based data
- Physical and Virtual Worlds: Social Networks and Social Media data & analysis

# Cloud-based Trends

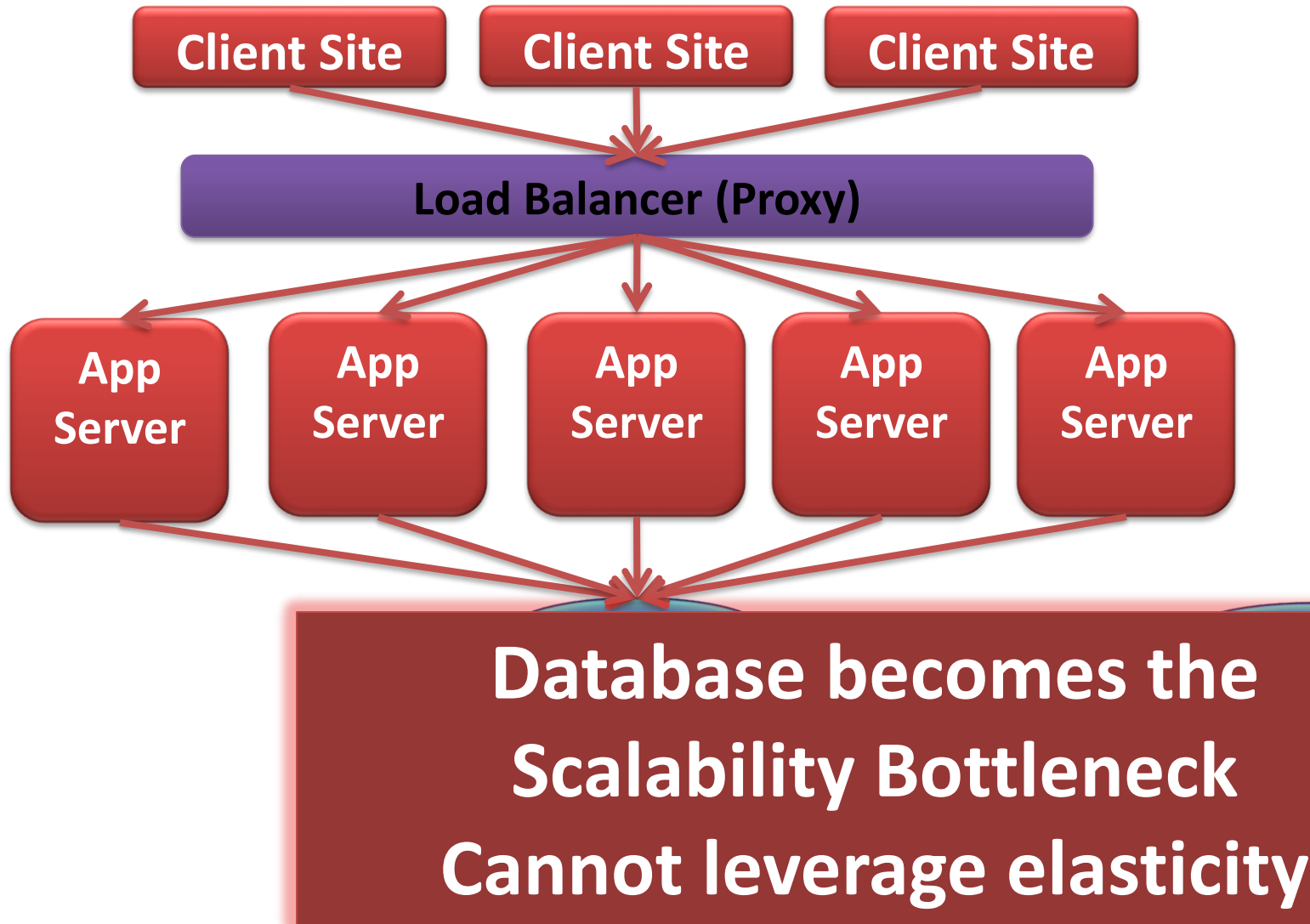
- New variety of Analysts:
  - Information-savvy users
  - Most users will become nimble analysts
  - Most transactional decisions will be preceded by a detailed analysis
- Online transaction processing, or **OLTP**, refers to a class of systems that facilitate and manage transaction-oriented applications, typically for data entry
- Convergence of OLAP (Online Analytical Processing) and OLTP:
  - Both from the application point-of-view and from the infrastructure point-of-view

# Platforms for Update intensive workloads

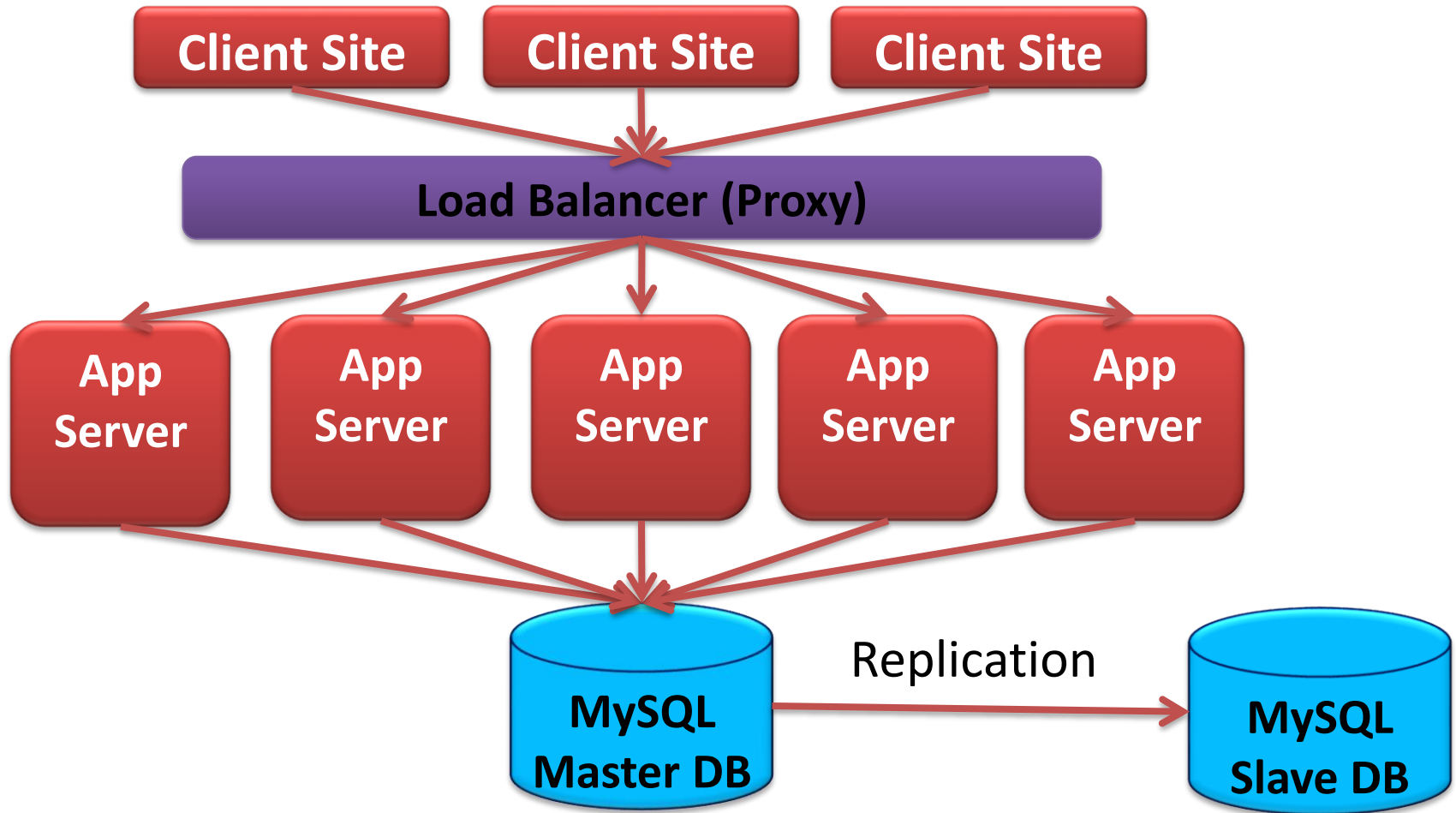
- Most enterprise solutions are based on RDBMS technology.
- Significant Operational Challenges:
  - Provisioning for Peak Demand
  - Resource under-utilization
  - Capacity planning: too many variables
  - Storage management: a massive challenge
  - System upgrades: extremely time-consuming
  - Complex mine-field of software and hardware licensing
- ➔ Unproductive use of people-resources from a company's perspective



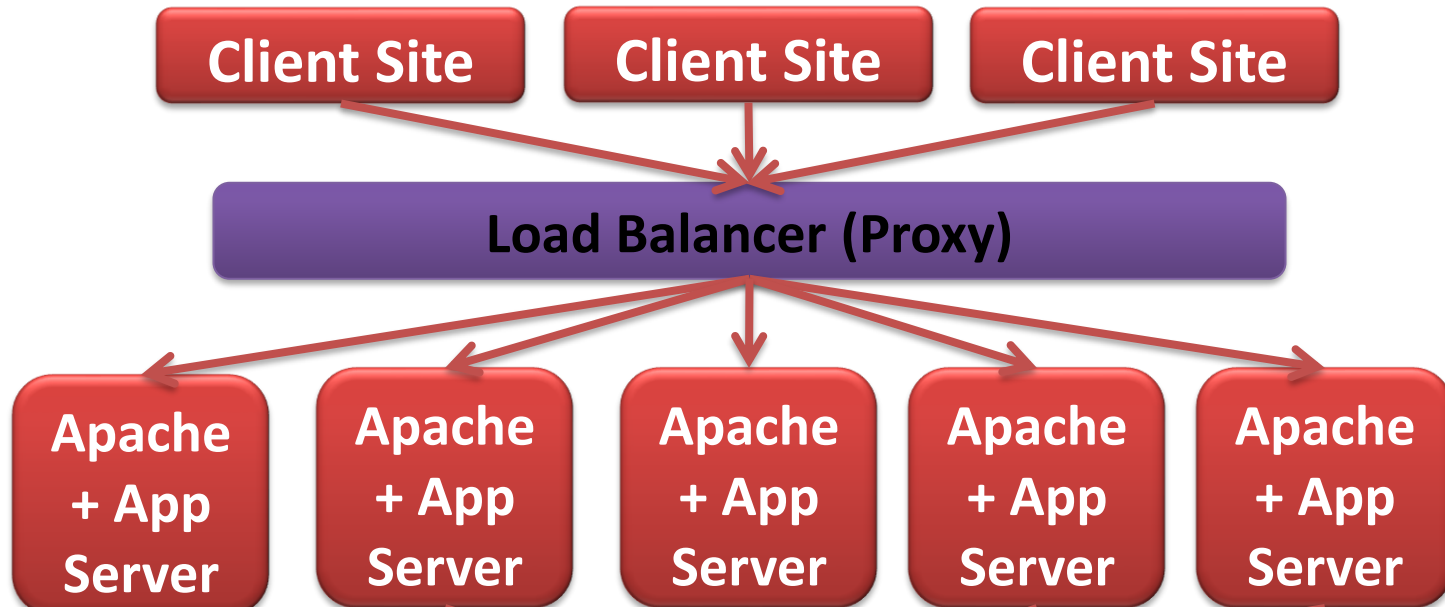
# Scaling in the Cloud



# Scaling in the Cloud



# Scaling in the Cloud



**Scalable and Elastic,  
but limited consistency and  
operational flexibility**

# Scaling in the Cloud

- “If we want vast, on-demand scalability, you need a non-relational database.” Since scalability requirements:
  - Can change very quickly and,
  - Can grow very rapidly.
- Difficult to manage with a single in-house RDBMS server.
- RDBMS scale well:
  - When limited to a single node, **but**
  - Overwhelming complexity to scale on multiple server nodes.

# The “NoSQL”

- Initially used for: *“Open-Source relational database that did not expose SQL interface”*
- Popularly used for: *“non-relational, distributed data stores that often did not attempt to provide ACID guarantees”*
- Gained widespread popularity through a number of open source projects
  - HBase, Cassandra, Voldemort, MongoDB, ...
- Scale-out, elasticity, flexible data model, high availability

# NoSQL → Not Only SQL

- Encompass a broad category of “structured” storage solutions
  - RDBMS is a subset
  - Key Value stores
  - Document stores
  - Graph database

# Cloud Data Management Requirements

SQL or NoSQL

- Scalability
- Elasticity
- Fault tolerance
- Self Manageability
- **Sacrifice consistency?**

# Sacrificing Consistency

Increased Application Complexity

```
public void confirm_friend_request(user1, user2)
{
begin_transaction();
    update_friend_list(user1, user2, status.confirmed); //Palo Alto
    update_friend_list(user2, user1, status.confirmed); //London
end_transaction();
}
```



```
public void confirm_friend_request_A(user1, user2){  
    try{  
        update_friend_list(user1, user2, status.confirmed); //palo alto  
    }  
    catch(exception e) {  
        report_error(e);  
        return;  
    }  
    try{  
        update_friend_list(user2, user1, status.confirmed); //london }  
    catch(exception e) {  
        revert_friend_list(user1, user2);  
        report_error(e);  
        return;  
    }  
}
```

```

public void confirm_friend_request_B(user1, user2){
try{
    update_friend_list(user1, user2, status.confirmed); //palo alto  }
    catch(exception e){
        report_error(e);
        add_to_retry_queue(operation.updatefriendlist, user1, user2,
            current_time());
    }
try{
update_friend_list(user2, user1, status.confirmed); //london
    }
catch(exception e) {
    report_error(e);
    add_to_retry_queue(operation.updatefriendlist, user2, user1,
        current_time());
    }
}
18
}

```

/\* get\_friends() method has to reconcile results returned by get\_friends() because there may be data inconsistency due to a conflict because a change that was applied from the queue is contradictory to a subsequent change by the user. In this case, status is a bitflag and changes are merged and it is up to app developer to figure out what to do. \*/

```
public list get_friends(user1){
    list actual_friends = new list();
    list friends = get_friends();
    foreach (friend in friends){
        if(friend.status == friendstatus.confirmed) {
            actual_friends.add(friend);
        } else if((friend.status &= friendstatus.deleted)
            and !(friend.status &= friendstatus.confirmed)){
            // assume friend is confirmed as long as it wasn't also deleted
            friend.status = friendstatus.confirmed;
            actual_friends.add(friend);
            update_friend(user1, friend, status.confirmed);
        } else { //assume there is a conflict with a delete
            update_friend(user1, friend, status.deleted);
        }
    }
} //foreach return actual_friends;
```

**It gets too complicated with Eventual Consistency**

Quest for **Scalable**, **Fault-tolerant**, and  
**Consistent** Data Management in the  
Cloud that provides  
**ELASTICITY**

# Key Value Stores

- Key-Valued data model
  - Key is the unique identifier
  - Key is the granularity for consistent access
  - Value can be structured or unstructured
- Gained widespread popularity
  - In house: **Bigtable** (Google), **PNUTS** (Yahoo!), **Dynamo** (Amazon)
  - Open source: **HBase**, **Hypertable**, **Cassandra**, **Voldemort**
- Popular choice for the modern breed of web-applications

# Key Value Stores

- Key Value Store example

| Students |                                                                                                                                                        |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Key      | Value                                                                                                                                                  |
| 1        | Name: Jean Grey<br>DateOfBirth: 19-05-1963<br>IDCard: 1234567<br>PlaceOfOrigin: Austin<br>Country: USA<br>AcademicProgram_ID:1                         |
| 2        | Name: Scott Summers<br>DateOfBirth: 12-10-1968<br>IDCard: 765414A<br>Supervisor: {<br>Name: Emma Frost<br>DateOfBirth: 1-1-1936<br>IDCard: 222222<br>} |

| Professors |                                                                                                             |
|------------|-------------------------------------------------------------------------------------------------------------|
| Key        | Value                                                                                                       |
| 1          | Name: Charles Xavier<br>DateOfBirth: 13-07-1940<br>IDCard: 111111<br>PlaceOfOrigin: Mirfield<br>Country: UK |
| 2          | Name: Emma Frost<br>DateOfBirth: 1-1-1936<br>IDCard: 222222                                                 |

# Key Value Stores

Key Value Store types

↑ — Increasing Data Complexity  
↓ —

| Type              | Examples                                                                                                                                                                    |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Key-Value Store   |       |
| Wide Column Store |       |
| Document Store    |    |
| Graph Store       |   |

# Important Design Goals

- **Scale out: designed for scale**
  - Commodity hardware
  - Low latency updates
  - Sustain high update/insert throughput
- **Elasticity – scale up and down with load**
- **High availability – downtime implies lost revenue**
  - Replication (with multi-mastering)
  - Geographic replication
  - Automated failure recovery



# Lower Priorities

- **No Complex querying functionality**
    - No support for SQL
    - CRUD operations through database specific API
  - **No support for joins**
    - Materialize simple join results in the relevant row
    - Give up normalization of data?
  - **No support for transactions**
    - Most data stores support single row transactions
    - Tunable consistency and availability (e.g., Dynamo)
- ➔ **Achieve high scalability**

# Interplay with CAP

- **C**onsistency, **A**vailability, and Network **P**artitions
  - Only have two of the three together
- Large scale operations – be prepared for network partitions
- Role of CAP – During a network partition, choose between Consistency and Availability
  - RDBMS choose *consistency*
  - Key Value stores choose *availability* [low replica consistency]

# Why sacrifice Consistency?

- It is a simple solution
  - nobody understands what sacrificing P means
  - sacrificing A is unacceptable in the Web
  - possible to push the problem to app developer
- C not needed in many applications
  - Banks do not implement ACID (classic example wrong)
  - Airline reservation only transacts reads (Huh?)
  - MySQL et al. ship by default in lower isolation level
- Data is noisy and inconsistent anyway
  - making it, say, 1% worse does not matter

# Network Partition

- Dynamo – quorum based replication
  - Multi-mastering keys – Eventual Consistency
  - Tunable read and write quorums
  - Larger quorums – higher consistency, lower availability
  - Vector clocks to allow application supported reconciliation
- PNUTS – log based replication
  - Similar to log replay – reliable log multicast
  - Per record mastering – timeline consistency
  - Major outage might result in losing the tail of the log

# Choosing among NoSQL Systems

Too many choices – Which system should I use?

The answer is to use common benchmarking to select among these choices.

# Benchmarking Serving Systems

[Cooper et al., SOCC 2010]

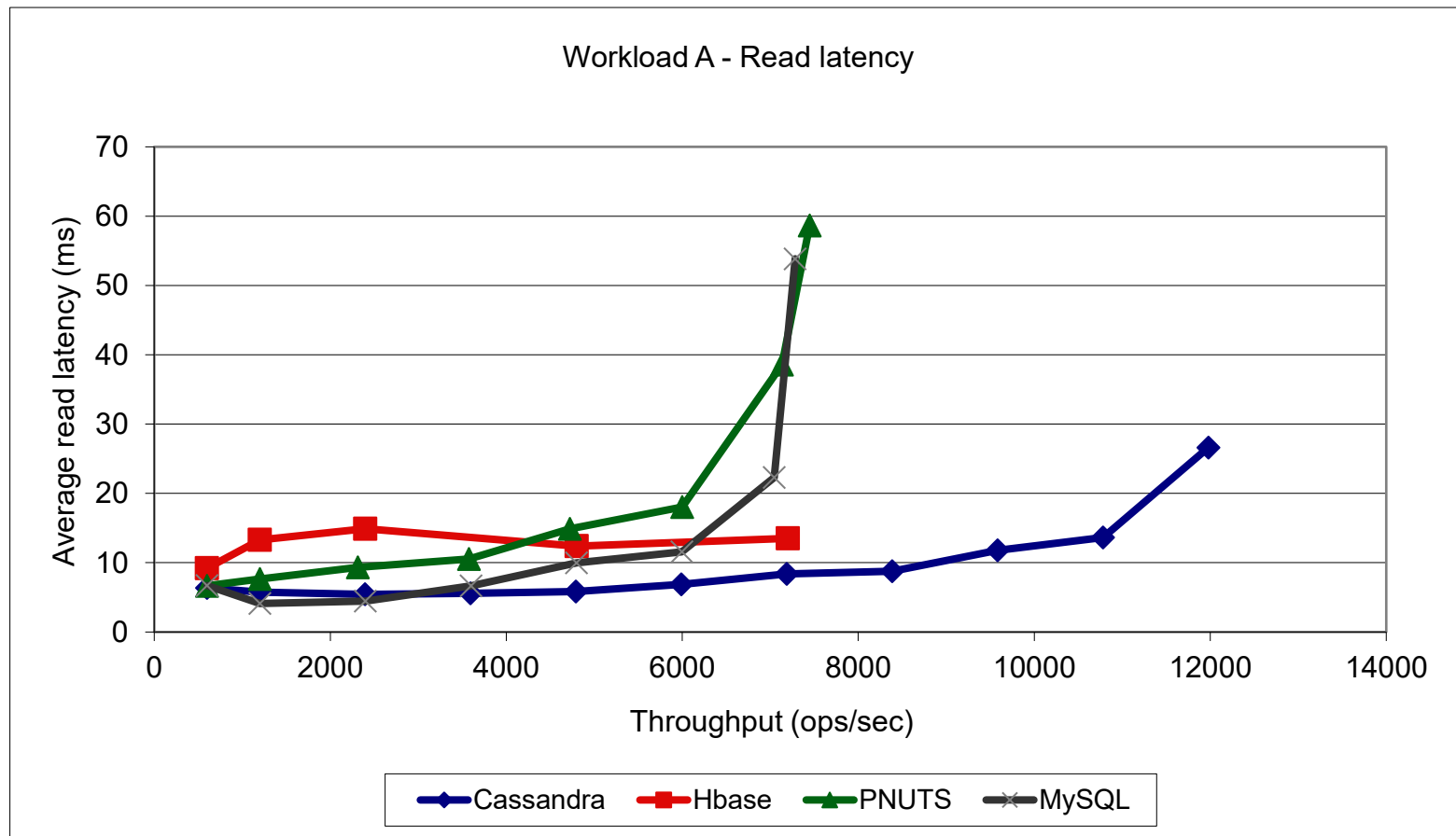
- A standard benchmarking tool for evaluating Key Value stores
- Evaluate different systems on common workloads
- Focus on performance and scale out

# Benchmark tiers

- Tier 1 – Performance
  - Latency versus throughput as throughput increases
- Tier 2 – Scalability
  - Latency as database, system size increases
  - “Scale-out”
  - Latency as we elastically add servers
  - “Elastic speedup”

# Workload A – Update heavy

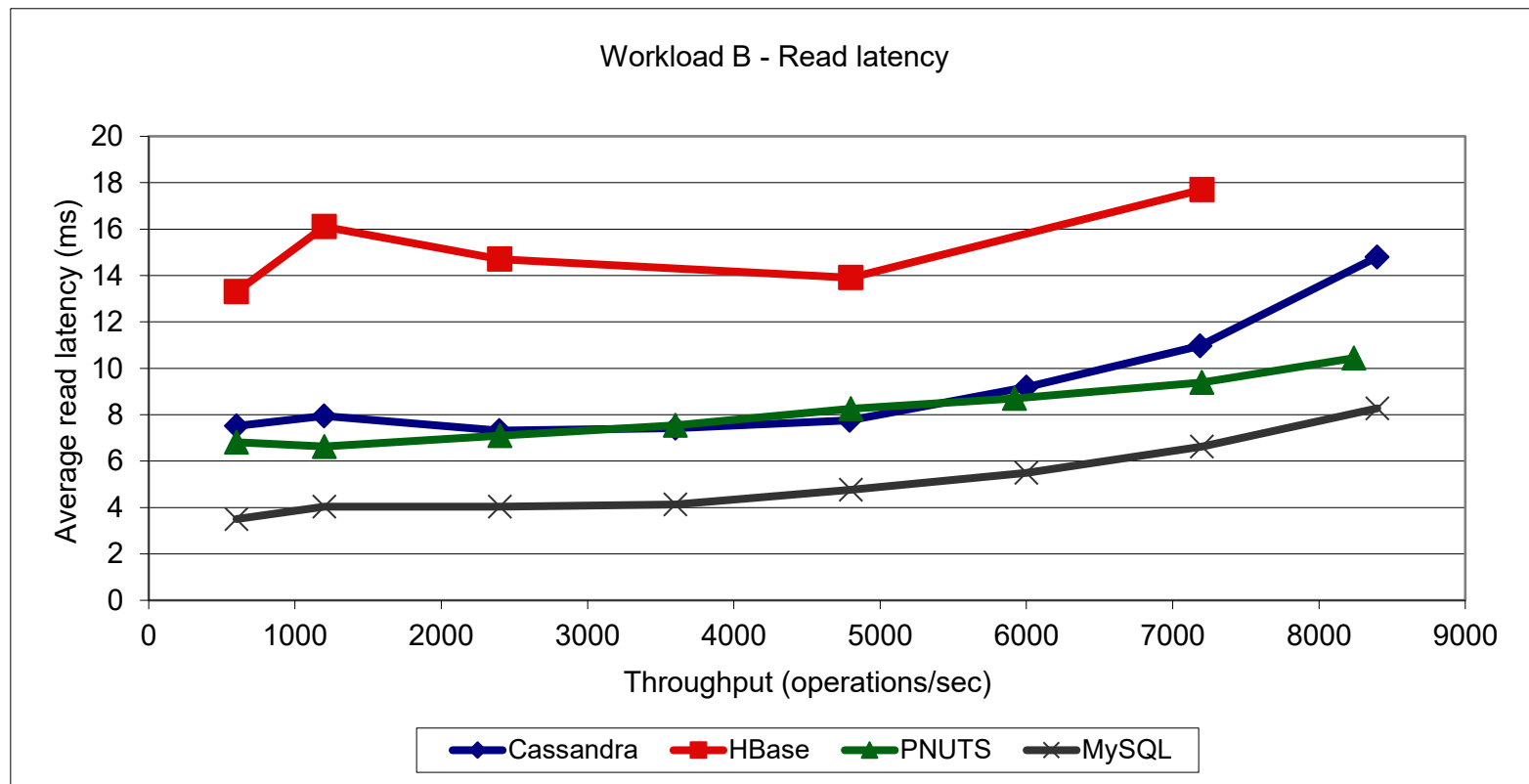
- 50/50 Read/update





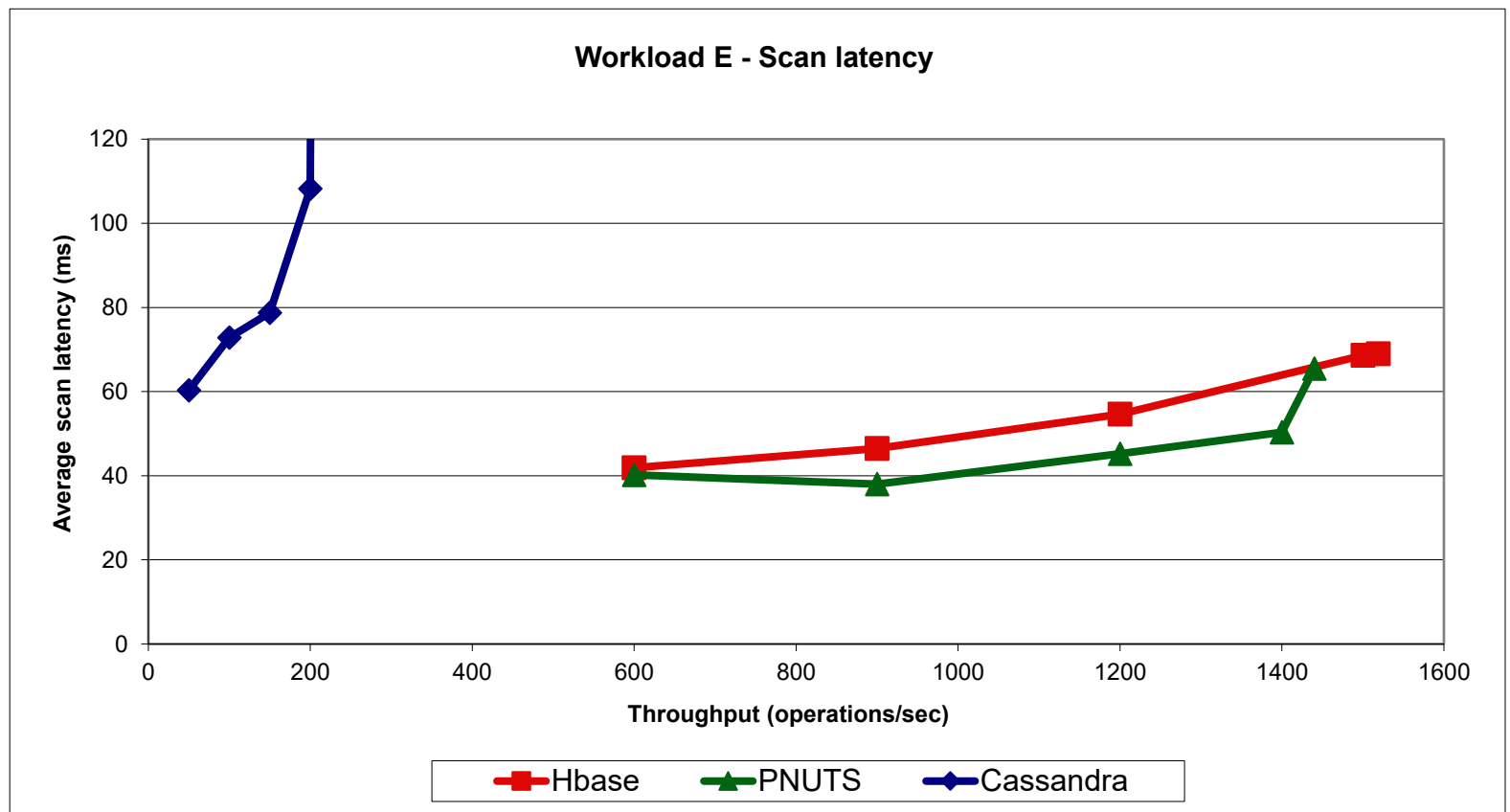
# Workload B – Read heavy

- 95/5 Read/update



# Workload E – short scans

- Scans of 1-100 records of size 1KB



# Summary

- Different databases suitable for different workloads
- Evolving systems – landscape changing dramatically
- Active development community around open source systems

# References

- **[Dean et al., OSDI 2004]** MapReduce: Simplified Data Processing on Large Clusters, J. Dean, S. Ghemawat, In OSDI 2004
- **[Dean et al., CACM 2008]** MapReduce: Simplified Data Processing on Large Clusters, J. Dean, S. Ghemawat, In CACM Jan 2008
- **[Dean et al., CACM 2010]** MapReduce: a flexible data processing tool, J. Dean, S. Ghemawat, In CACM Jan 2010
- **[Stonebraker et al., CACM 2010]** MapReduce and parallel DBMSs: friends or foes?, M. Stonebraker, D. J. Abadi, D. J. DeWitt, S. Madden, E. Paulson, A. Pavlo, A. Rasin, In CACM Jan 2010
- **[Pavlo et al., SIGMOD 2009]** A comparison of approaches to large-scale data analysis, A. Pavlo et al., In SIGMOD 2009
- **[Abouzeid et al., VLDB 2009]** HadoopDB: An Architectural Hybrid of MapReduce and DBMS Technologies for Analytical Workloads, A. Abouzeid et al., In VLDB 2009
- **[Afrati et al., EDBT 2010]** Optimizing joins in a map-reduce environment, F. N. Afrati, J. D. Ullman, In EDBT 2010
- **[Agrawal et al., SIGMOD 2009]** Asynchronous view maintenance for VLSD databases, P. Agrawal et al., In SIGMOD 2009
- **[Das et al., SIGMOD 2010]** Ricardo: Integrating R and Hadoop, S. Das et al., In SIGMOD 2010
- **[Cohen et al., VLDB 2009]** MAD Skills: New Analysis Practices for Big Data, J. Cohen et al., In VLDB 2009

- **These slides are based on the presentation:**

**Big Data and Cloud Computing: Current State and Future Opportunities.** EDBT 2011 Tutorial. *Divy Agrawal, Sudipto Das, and Amr El Abbadi.*  
*Department of Computer Science, University of California at Santa Barbara*